CineHub - Movie Booking Platform

Author- Dinesh Metkari

Contents

1. Overview 4
2 Functional Requirements: 5
2.1 User Management 5
2.2 Movie & Show Browsing 5
2.3 Theatre & Screen management 5
2.4 Seat selection 5
2.5 Booking & Ticketing 5
2.6 Booking platform offers 5
3 Nonfunctional Requirements 6
3.1 Performance 6
3.2 Scalability 6
3.3 Availability 6
3.4 Data consistency 6
3.5 Maintainability 6
3.6 Localization 6
3.7 Logging & Monitoring 6
4 How to run 7
5 Technology Stack 7
6 Architecture 9
7 Design patterns 12
8 Key design decisions 13
9 Data model 14
10 Platform Provisioning 16
11 Getting started - Configuration 19
12 API reference 21
a. Authentication — /api/v1/auth 21
b. Customer — /api/v1/customer 21
c. Theatre partner — /api/v1/theatre 23
13 Configuration reference 25

14 Future enhancements 26

1. Overview

A B2B/B2C movie ticket booking platform built with Java and Spring Boot.

Users	Operation
Theatre Partners (B2B)	Register theatres, create and manage shows, control seat inventory, view bookings
Customers (B2C)	Browse movies by city/genre/language, view show timings, book seats, apply offers, cancel bookings

2 Functional Requirements:

2.1 User Management

-Register

-Login

2.2 Movie & Show Browsing

-Users can browse movies by city, language, release date

-User can view movie details

-User can view available showtimes for a selected movie in a particular theatre

2.3 Theatre & Screen management

- Admin can add/edit/delete theatres, screen and seat map.

2.4 Seat selection

-User can view the seat map for a selected showtimes

-Users can select available seats and proceed to booking

2.5 Booking & Ticketing

2.6 Booking platform offers

- 50% discount applied for third ticket

- Ticket booked for afternoon show get 20% discount

3 Nonfunctional Requirements

3.1 Performance

-The system should support at least 10000 concurrent users without noticeable latency.

3.2 Scalability

System should be horizontally scalable to handle high traffic especially during blockbuster releases.

3.3 Availability

System should ensure 99.9% uptime

3.4 Data consistency

Booking transactions must ensure ACID properties

3.5 Maintainability

Codebase should be modular and easy for maintenance and extension

3.6 Localization

The system should support multiple languages

3.7 Logging & Monitoring

All operations should be logged for audit and debugging.

System health should be monitored.

4 How to run

Prerequisites

- Java 20+
- MySQL 8+
- Maven 3.8+
- Spring Boot 3+
- Git
- Docker

Open command prompt; c:

Clone Github project:

git clone https://github.com/dineshmetkari/CineHub

cd CineHub

docker-compose up -d

Open browser & open url:

http://localhost:8080/swagger-ui/index.html

-Register

- Login

-Copy token to Authorize

-Explore API endpoints.

5 Technology Stack

Concern	Choice
Language	Java 20

Concern	Choice
Framework	Spring Boot 3.2.1
Database	MySQL 8 (H2 in-memory for tests)
ORM	Spring Data JPA / Hibernate
Security	Spring Security + JWT (jjwt 0.12.3)
API Docs	SpringDoc OpenAPI 3 (Swagger UI)
Build	Maven 3.8+
Connection pool	HikariCP (20 max connections)
Cache	Spring Cache (simple in-memory) Assumption: Redis in production
Docker	Container

6 Architecture

```
┌──────────────────────────────────────────────────────────────┐
│                        Client Layer                          │
│          (Web browser, Mobile app, Postman / curl)           │
└──────────────────────────────────────────────────────────────┘
                              │
                              ▼
┌──────────────────────────────────────────────────────────────┐
│               Spring Security — JWT Filter Chain             │
│   JwtAuthenticationFilter  →  SecurityContextHolder          │
└──────────────────────────────────────────────────────────────┘
                              │
              ┌───────────────┼───────────────┐
              ▼               ▼               ▼
      ┌──────────────┐ ┌──────────────┐ ┌──────────────┐
      │ AuthController│ │  Customer    │ │  Theatre     │
      │ /api/v1/auth  │ │  Controller  │ │  Controller  │
      │               │ │/api/v1/      │ │ /api/v1/     │
      │               │ │customer      │ │ theatre      │
      └──────────────┘ └──────────────┘ └──────────────┘
              │               │               │
              └───────────────┼───────────────┘
                              ▼
      ┌──────────────────────────────────────────────┐
      │                  Service Layer               │
      ├──────────────────────────────────────────────┤
      │  AuthService       ShowService               │
      │  BookingService    OfferService              │
      │  UserResolverService                         │
      │                                              │
      │  strategy/                                   │
      │    AfternoonShowDiscountStrategy             │
      │    ThirdTicketDiscountStrategy               │
      └──────────────────────────────────────────────┘
                              │
      ┌──────────────────────────────────────────────┐
      │               Repository Layer               │
      ├──────────────────────────────────────────────┤
      │  UserRepository        TheatreRepository     │
      │  MovieRepository       ShowRepository        │
      │  ScreenRepository      SeatRepository        │
      │  BookingRepository     OfferRepository       │
      │  CityRepository                              │
      └──────────────────────────────────────────────┘
                              │
                              ▼
                   ┌──────────────────┐
                   │   MySQL 8        │
                   │  (HikariCP 20)   │
                   └──────────────────┘
```

Layer responsibilities

Controller layer — three controllers, all under `/api/v1`:

Controller	Prefix	Auth required
`AuthController`	`/auth`	None (public)
`CustomerController`	`/customer`	JWT for booking endpoints; browse/offers are public
`TheatreController`	`/theatre`	JWT with `ROLE_THEATRE_PARTNER` on every endpoint

Controllers extract the authenticated user via `@AuthenticationPrincipal UserDetails` and delegate to `UserResolverService.resolveCurrentUserId()` to get the database `Long` userId. No `?partnerId=` or `?customerId=` query params — identity always comes from the token.

Service layer — all business logic, validation, and transaction boundaries live here. Controllers never call repositories directly.

Repository layer — one `JpaRepository` per entity, with named JPQL queries for complex lookups. No native SQL; everything goes through the ORM.

Package structure

```
src/main/java/com/cinehub/booking/
├── config/          # SecurityConfig, OpenApiConfig, AuditConfig, DataSeeder
├── controller/      # AuthController, CustomerController, TheatreController
├── dto/
│   ├── request/     # AuthRequest, BookTicketsRequest, CreateShowRequest, BulkCancelRequest
│   └── response/    # AuthResponse, BookingResponse, ShowDetailsResponse,
│                    # PriceCalculationResponse, SeatResponse
├── entity/          # City, User, Theatre, Screen, Movie, Show, Seat, BookingSeat, Booking, Offer
├── enums/           # UserRole, BookingStatus, SeatStatus, ShowTimeSlot
├── exception/       # ResourceNotFoundException, BookingException,
│                    # InvalidOperationException, GlobalExceptionHandler
├── repository/      # One JpaRepository per entity, with custom JPQL queries
├── security/        # JwtTokenProvider, JwtAuthenticationFilter, UserDetailsServiceImpl
└── service/
    ├── strategy/    # DiscountStrategy (interface), AfternoonShowDiscountStrategy,
    │                # ThirdTicketDiscountStrategy
```

```
    ├── AuthService.java
    ├── BookingService.java
    ├── OfferService.java
    ├── ShowService.java
    └── UserResolverService.java
src/main/resources/
└── application.yml     # MySQL datasource, JWT config, app.offers.* properties, test profile


src/test/java/com/cinehub/booking/
└── BookingWorkflowTest.java
```

7 Design patterns

Strategy Pattern — discount calculation Offer Service holds a List<DiscountStrategy> injected by Spring. At booking time every strategy's isApplicable() is checked and all matching discounts are **stacked** — both the afternoon and third-ticket discounts can fire together on the same booking. Adding a new offer requires only a new @Component class; no existing code changes.

```
DiscountStrategy (interface)

  ├── AfternoonShowDiscountStrategy  — 20% off shows 12:00–16:00

  └── ThirdTicketDiscountStrategy    — 50% off every 3rd ticket (price-desc sort)
```

Repository Pattern All data access is behind JpaRepository interfaces with named JPQL queries. Business logic in services never builds queries inline.

DTO Pattern Separate request and response classes for every API surface. Entities never leave the service layer.

8 Key design decisions

1. Concurrency — pessimistic + optimistic locking combined

Problem: Multiple customers booking the same seat simultaneously leads to double-booking.

Solution — two layers:

Layer 1: Pessimistic write lock on seats (SeatRepository)

```
@Lock(LockModeType.PESSIMISTIC_WRITE)

@Query("SELECT s FROM Seat s WHERE s.seatId IN :seatIds")

List<Seat> findSeatsByIdWithPessimisticLock(@Param("seatIds") List<Long> seatIds);
```

When bookSelectedTickets() begins, a SELECT … FOR UPDATE is issued on the requested seats. Any concurrent transaction requesting the same seat IDs blocks at the database level until the first transaction commits or rolls back. The second transaction then re-evaluates availability and rejects the already-booked seats.

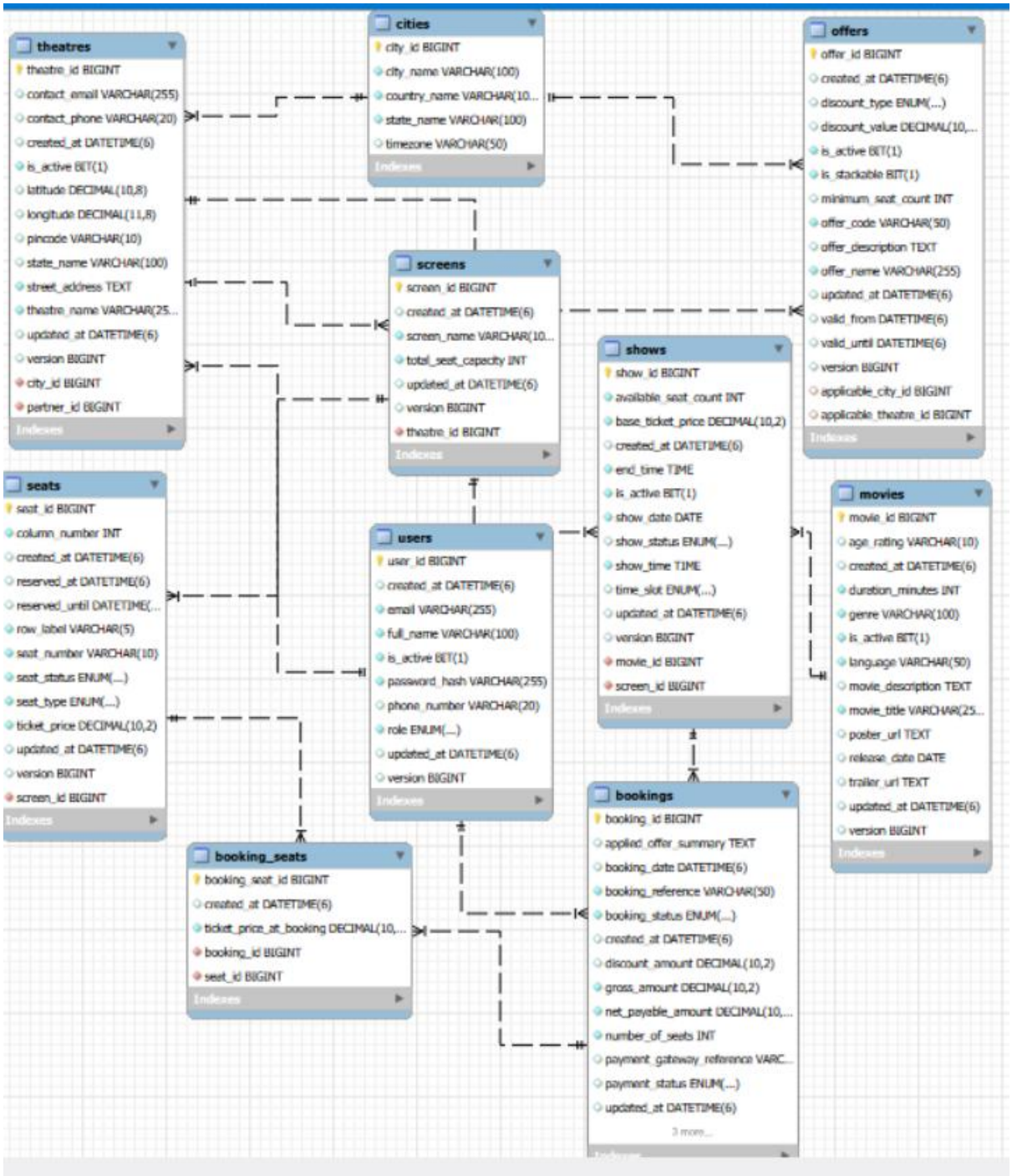
2. Authentication — stateless JWT

Every request carries a Bearer token in the Authorization header. JwtAuthenticationFilter intercepts each request before the controller, validates the token via JwtTokenProvider (HS256, jjwt 0.12.3), and populates SecurityContextHolder. Sessions are never created (SessionCreationPolicy.STATELESS).

The partner identity is extracted from the token — no ?partnerId= query param is accepted. UserResolverService.resolveCurrentUserId() converts the email subject from the JWT to the database Long userId on every authenticated call.

9 Data model

```
City —< Theatre —< Screen —< Seat
                  |
                 Show >— Movie
                  |
                Booking —< BookingSeat >— Seat
                  |
                 User
```



Key entity decisions:

- **Pessimistic write lock** (PESSIMISTIC_WRITE) on the seat query inside BookingService prevents two concurrent transactions from booking the same seat simultaneously.

10 Platform Provisioning

- **Cloud Provider:**

  AWS / Azure / GCP

- **Compute:**

  Kubernetes (EKS) / Docker containers

- **Networking:**

  VPC with:

  Public Subnet (Load balancer)

  Private Subnet (services, DB)

- **Environment Setup:**

  Separate environments for stability and release control:

  Environment Purpose

  Dev Developer testing

  QA Integration testing

  Staging Production-like

  Prod Live system

- **Environment Config**

  Config Server (Spring Cloud Config)

- **Environment variables**

  Secrets management (AWS Secrets Manager / Vault)

- **Containerization & Orchestration**

  Docker

- **CI/CD Pipeline Provisioning**

  Automates build & deployment.

- **Tools**

  GitHub Actions / Jenkins / GitLab CI/Azure Pipeline

- **Pipeline Stages**

  Code checkout

  Build (Maven/Gradle)

  Unit Tests

  Docker image build

  Push to registry (Docker Hub / ECR)

  Deploy to Kubernetes

- **Database Provisioning**

  Primary DB

  MySQL / PostgreSQL (RDS)

- **Configuration**

  Multi-AZ deployment

- **Automated backups**
- **Read replicas (for scaling)**
- **Caching Layer**

  Redis (critical for seat locking)

- **Monitoring**

  Metrics: Prometheus

  Visualization: Grafana

  Logging: ELK Stack

  Cloud-native: AWS CloudWatch

- **Security Layers**

  Authentication

  - JWT-based login
  - Token passed in headers

  Authorization

  - Role-based access:

    - USER → book tickets
    - ADMIN → manage theatres

- **API Gateway Security**

  Validate JWT centrally

  Rate limiting

- **Data Security**

  HTTPS (TLS)

  Encrypt DB credentials

  Mask sensitive logs

- **Auto-Scaling**

  Orchestration: Kubernetes

  Scaling: Horizontal Pod Autoscaler

  Cloud scaling: AWS Auto Scaling

- **Scaling Strategy**

  Horizontal Scaling (preferred)

  - Increase number of pods

  Vertical Scaling

  - Increase CPU/RAM (less preferred)

11 Getting started - Configuration

1 — Create the MySQL database

CREATE DATABASE movie_booking CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;

2 — Configure credentials

Edit src/main/resources/application.yml:

```
spring:
  datasource:
    url: jdbc:mysql://localhost:3306/movie_booking?useSSL=false&serverTimezone=UTC&allowPublicKeyRetrieval=true
    username: your_username
    password: your_password
```

3 — Configure the JWT secret

Replace the default secret with a strong random string of at least 32 characters:

```
jwt:
  secret: your-strong-secret-key-at-least-32-characters
  expiration-ms: 86400000   # 24 hours
```

4 — Build and run

```
mvn clean install
mvn spring-boot:run
```

The application starts on http://localhost:8080.
On first startup DataSeeder runs automatically and populates the database with demo cities, users, theatres, screens, seats, movies, and shows.

5 — Swagger UI

http://localhost:8080/swagger-ui.html

Use **Authorize** (top right) and paste the JWT token from the login response as Bearer <token>.

Demo seed data

DataSeeder creates the following on first startup (idempotent — skipped if data exists):

Type	Details
Cities	Mumbai, Bangalore, Delhi
Users	customer@example.com, partner@example.com, admin@example.com (password: Password@123)
Theatres	PVR Cinemas — Lower Parel (Mumbai), INOX — Koramangala (Bangalore)
Screens	2 screens per theatre (Screen 1: 100 seats, Screen 2: 80 seats)
Seats	Rows A–C → PREMIUM (₹300), Rows D–J → REGULAR (₹200)
Movies	The Great Adventure (Hindi/Action), Comedy Nights (Hindi/Comedy), The Silent Thriller (English/Thriller)
Shows	Morning (10:00), **Afternoon (14:00)**, Evening (18:00), Night (21:30) — today and tomorrow

The 14:00 show is in the afternoon window so both discount strategies fire immediately when test with 3+ seats.

12 API reference

a. Authentication — /api/v1/auth

Method	Path	Body	Description
POST	/register	{ fullName, email, password, phoneNumber, role }	Register (role: CUSTOMER or THEATRE_PARTNER)
POST	/login	{ email, password }	Returns { accessToken, tokenType, userId, fullName, role }

b. Customer — /api/v1/customer

All browse endpoints are public. Booking endpoints require Authorization: Bearer <token>.

Method	Path	Query params	Description
GET	/movies	city, language, genre (all optional)	Browse active movies
GET	/movies/{movieId}/shows	city, date	**Read Scenario 1** — theatres and show timings for a movie in a city on a date
GET	/shows/{showId}/seats	—	Available seats for a show (seat grid)

Method	Path	Query params	Description
POST	/offers/calculate	showId, seatIds	**Read Scenario 2** — preview pricing with all applicable discounts
POST	/bookings	—	**Write Scenario 1** — book tickets (discounts applied automatically)
POST	/bookings/bulk	—	Bulk book — many seats for one show in one transaction
GET	/bookings	—	My booking history
GET	/bookings/{reference}	—	Booking detail by reference number
DELETE	/bookings/{reference}	—	Cancel a booking
DELETE	/bookings/bulk	—	Bulk cancel

Book tickets request body:

```
{
  "showId": 1,
  "selectedSeatIds": [10, 11, 12],
  "promoCode": "optional"
```

}

c. Theatre partner — /api/v1/theatre

All endpoints require Authorization: Bearer <token> with role THEATRE_PARTNER.

Method	Path	Description
POST	/shows	**Write Scenario 2** — create a show (checks for scheduling conflicts)
PUT	/shows/{showId}	Update show date, time, or price (blocked if bookings exist)
DELETE	/shows/{showId}	Cancel a show (blocked if bookings exist)
GET	/shows	View shows at a theatre on a date (?theatreId=&date=)
GET	/shows/{showId}/seats	View full seat layout and statuses
PUT	/shows/{showId}/seats	**Write Scenario 3** — bulk update seat status (?seatIds=&newStatus=BLOCKED)
GET	/shows/{showId}/bookings	View all bookings for a show

Create show request body:

```
{
  "movieId": 1,
  "screenId": 1,
  "showDate": "2026-03-20",
  "showTime": "14:30",
  "baseTicketPrice": 250.00
}
```

Offer implementation

Both offers are configured in application.yml and applied automatically — no promo code required:

```
app:
  offers:
    third-ticket-discount-percent: 50
    afternoon-discount-percent: 20
    afternoon-start-hour: 12
    afternoon-end-hour: 16
```

Third ticket discount (50% off every 3rd seat) Seats are sorted by price descending before applying the discount so the highest-priced 3rd seat always gets discounted.

Example — 5 seats [₹300, ₹300, ₹300, ₹200, ₹200] sorted desc:

- Position 3: ₹300 → 50% off = **₹150 discount**
- Position 6: does not exist
- Total discount: ₹150

Afternoon show discount (20% off total) Applies when showTime >= afternoonStartHour and showTime < afternoonEndHour.

Stacking Both discounts fire independently and are **added together**. A 3-ticket booking at 14:00 gets both:

Gross: ₹900 (3 × ₹300)

3rd seat: −₹150 (50% of ₹300)

Afternoon:−₹180 (20% of ₹900)

Net: ₹570

13 Configuration reference

Property	Default	Description
jwt.secret	(set this)	HS256 signing key — minimum 32 characters
jwt.expiration-ms	86400000	Token lifetime (24 hours)
app.offers.third-ticket-discount-percent	50	% off every 3rd ticket
app.offers.afternoon-discount-percent	20	% off afternoon shows
app.offers.afternoon-start-hour	12	Afternoon window start (inclusive)
app.offers.afternoon-end-hour	16	Afternoon window end (exclusive)
spring.jpa.hibernate.ddl-auto	update	Use validate in production with Flyway

14 Future enhancements

- Payment gateway integration (Razorpay / Stripe)
- Email and SMS booking confirmations
- Seat hold with expiry timer (reserve for 10 minutes before payment)
- Redis cache for movie catalogue and show schedules
- Analytics dashboard for theatre partners
- Microservices and Kafka implementation
- Deployment on EKS
- Terraform for DevOps